

Progetto Web Service API

Pianificatore pasti settimanali

Benedettini Marco

Macchia Francesco

Randazzo Kevin

Indice

1	Introduzione	3
2	Obiettivi	3
3	Architettura del sistema	3
3.1	Frontend	4
3.2	Backend	4
3.3	Database locale delle ricette	4
3.4	API esterna – USDA FoodData Central	5
3.5	Tabella dei componenti	5
4	Database SQL	5
4.1	Schema	5
4.2	Normalizzazione degli ingredienti	5
5	Importazione dati	6
6	Protocollo	6
7	Risposta JSON	6
7.1	Struttura radice (HTTP 200)	6
7.2	Campo summary	7
7.3	Elemento di plan	7
7.4	Elemento di meals	7
8	Componenti del backend	7
8.1	Flusso di una richiesta a runtime	7
9	Logica no ripetizioni	8
9.1	Distribuzione delle calorie	8
10	Routing con ModRewrite	9
11	Testing	9
11.1	cURL	9
11.2	Postman	9

1 Introduzione

Il progetto consiste nella realizzazione di un sistema Web Server per la pianificazione automatica di piani alimentari settimanali personalizzati. Il sistema espone una API RESTful scritta interamente in PHP che, ricevute le preferenze dell'utente (calorie giornaliere, intolleranze, giorni), restituisce un piano pasti fino a sette giorni evitando la ripetizione di ricette e ingredienti principali.

Le fonti dati utilizzate sono due e hanno ruoli distinti:

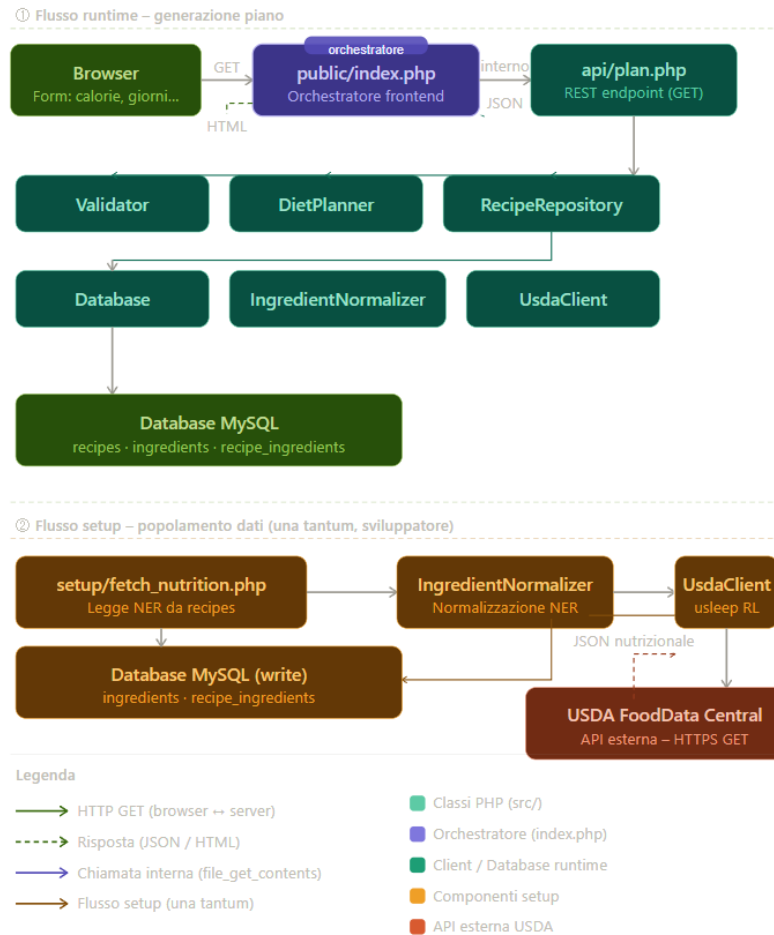
- **Dataset Kaggle - Recipe Dataset (2M+)**: raccolta di oltre due milioni di ricette con titolo, elenco ingredienti in testo libero e istruzioni di preparazione. Costituisce il catalogo di ricette del sistema.
- **USDA FoodData Central (FDC)**: servizio REST ufficiale del Dipartimento dell'Agricoltura degli Stati Uniti che fornisce valori nutrizionali (calorie, proteine, grassi, carboidrati, fibre, zuccheri, grassi saturi, ecc..) per centinaia di migliaia di alimenti. Viene interrogato una sola volta per ingrediente durante la fase di importazione; i risultati vengono salvati localmente per eliminare qualsiasi dipendenza dall'API a runtime.

2 Obiettivi

- Ricevere le preferenze dell'utente tramite HTTP (calorie target, intolleranze, numero di giorni, presenza di uno snack).
- Generare un piano senza la ripetizione di ricette né di ingredienti principali.
- Restituire la risposta al client in formato JSON.
- Interfaccia HTML e PHP per la visualizzazione del piano nel browser.
- Utilizzare USDA FDC come API REST esterna per arricchire il database locale di nutrienti durante la fase di setup.
- Testing con cURL e Postman.

3 Architettura del sistema

L'architettura è organizzata in quattro livelli che comunicano tramite HTTP e accessi SQL.



3.1 Frontend

Una pagina PHP con form HTML raccoglie i parametri dell'utente e li invia al backend tramite una richiesta GET. La risposta JSON ricevuta viene poi formattata e mostrata nella stessa pagina.

3.2 Backend

Un server Apache gestito da XAMPP che ospita il codice, Il backend si occupa di:

- validare e sanificare i parametri in ingresso;
- interrogare il database locale con query SQL filtrate per dieta, intolleranze e budget;
- applicare la logica di creazione del piano e nessuna ripetizione;
- restituire la risposta in formato JSON.

3.3 Database locale delle ricette

Un database MySQL contiene le ricette normalizzate e i valori nutrizionali pre-caricati. Tutta la logica di pianificazione legge esclusivamente da questo database: a runtime non avviene alcuna chiamata HTTP esterna, garantendo tempi di risposta di pochi secondi.

3.4 API esterna – USDA FoodData Central

USDA FDC viene interrogata esclusivamente durante la fase di importazione (setup), e mai a runtime. Per ogni ingrediente unico estratto dal dataset Kaggle, viene effettuata una singola chiamata all’endpoint di ricerca di FDC. I valori nutrizionali restituiti vengono salvati nel database locale con un TTL di 30 giorni, aggiornabile manualmente con il flag `–force-update`. Questo modo soddisfa il requisito di utilizzo di un servizio REST esterno mantenendo il sistema indipendente dai limiti del rate limit dell’API (1000 richieste all’ora) durante il normale funzionamento.

3.5 Tabella dei componenti

Componente	Tecnologia	Ruolo
Browser	HTML e PHP	Interfaccia utente finale
Server	PHP 8 e Apache (XAMPP)	Logica API, query SQL
Database locale	MySQL	Ricette e nutrienti
API esterna	USDA FDC - REST/HTTPS	Sorgente valori nutrizionali

Tabella 1: Componenti del sistema

4 Database SQL

4.1 Schema

Il database è composto da tre tabelle principali.

recipes – contiene le ricette importate dal dataset Kaggle: identificatore univoco, titolo, campo NER (JSON array dei nomi degli ingredienti) e tipo di pasto (breakfast, lunch, dinner, snack).

ingredients – funge da cache permanente dei valori nutrizionali ottenuti da USDA. Ogni riga corrisponde a un ingrediente normalizzato e contiene: calorie per 100 g, proteine, grassi, carboidrati, fibre, zuccheri e grassi saturi. Il campo source distingue i valori ottenuti dall’API (`'usda'`) da quelli stimati per categoria (`'estimated'`) quando USDA non restituisce risultati.

recipe_ingredients – tabella di collegamento molti-a-molti tra ricette e ingredienti. Ogni riga associa una ricetta a un ingrediente con la relativa quantità in grammi, necessaria al calcolo dei macro totali della ricetta.

4.2 Normalizzazione degli ingredienti

Gli ingredienti nel dataset Kaggle sono espressi come testo libero nel campo NER (ad esempio `fresh chopped onions`). La classe `IngredientNormalizer` rimuove aggettivi di stato e preparazione (`fresh`, `dried`, `chopped`, `sliced`), uniforma i plurali e abbassa tutto in minuscolo. Il risultato normalizzato (ad esempio `“onion”`) viene usato sia come chiave di ricerca su FDC sia come chiave univoca nella tabella `ingredients`, evitando duplicati e chiamate ridondanti.

5 Importazione dati

Lo script `fetch_nutrition.php` viene eseguito una sola volta prima della messa in produzione, oppure nuovamente con il flag `-force-update` per aggiornare i valori nutrizionali.

Il flusso è il seguente: le ricette vengono lette dalla tabella `recipes`. Per ogni ricetta vengono estratti gli ingredienti normalizzati dal campo `NER`. Se un ingrediente è già presente nella tabella `ingredients`, non viene effettuata alcuna chiamata API (salvo `-force-update`). In caso contrario viene interrogato USDA FDC e il risultato salvato localmente. Quando FDC non restituisce risultati per un ingrediente, vengono utilizzati valori medi stimati per categoria alimentare (carni, cereali, grassi, latticini, verdure), registrati con `source = 'estimated'`.

Un pool di 50 ricette contiene tipicamente 80–120 ingredienti unici normalizzati, ben al di sotto del limite orario di USDA.

6 Protocollo

Il sistema utilizza HTTP come protocollo di trasporto. I parametri vengono inviati tramite query string (metodo GET) e la risposta è sempre in formato JSON con `content-type application/json`.

Parametro	Tipo	Default	Note
calories	int	2000	Range 1000–5000
allergies	csv	—	Parole chiave separate da virgola (es. peanut,gluten)
days	int	7	Range 1–7
snacks	int	0	Se 1, aggiunge uno spuntino riducendo le quote degli altri pasti

Tabella 2: Parametri della richiesta

Il routing delle richieste è gestito tramite `mod_rewrite` di Apache, che mappa URL leggibili (ad esempio `/api/plan`) al controller PHP corrispondente.

7 Risposta JSON

7.1 Struttura radice (HTTP 200)

Campo	Tipo	Descrizione
success	boolean	Sempre true in caso di successo
inputs	object	Parametri ricevuti e validati
summary	object	Statistiche aggregate del piano generato
plan	array	Piano giorno per giorno

Tabella 3: Struttura radice della risposta

Campo	Tipo	Descrizione
days	number	Giorni pianificati
unique_recipes_used	number	Ricette distinte usate nel piano
avg_calories_per_day	number	Media kcal giornaliera

Tabella 4: Struttura del campo summary

7.2 Campo summary

7.3 Elemento di plan

Campo	Tipo	Descrizione
day	number	Numero del giorno (1–7)
meals	array	Pasti del giorno (colazione, [snack], pranzo, cena)
totals.calories	number	Somma kcal dei pasti
totals.protein_g	number	Somma proteine in grammi
totals.fat_g	number	Somma grassi in grammi
totals.carbs_g	number	Somma carboidrati in grammi
totals.fiber_g	number	Somma fibre in grammi
totals.sugar_g	number	Somma zuccheri in grammi
totals.saturated_fat_g	number	Somma grassi saturi in grammi

Tabella 5: Struttura di un giorno nell’array plan

7.4 Elemento di meals

Campo	Tipo	Descrizione
meal	string	breakfast / snack / lunch / dinner
recipe	string	Nome della ricetta
recipe_id	number	ID univoco nel database locale
portion_factor	number	Fattore di scala porzione (0.5–2.5)
calories	number	Calorie della porzione scalata
protein_g	number	Proteine in grammi
fat_g	number	Grassi in grammi
carbs_g	number	Carboidrati in grammi
fiber_g	number	Fibre in grammi
sugar_g	number	Zuccheri in grammi
saturated_fat_g	number	Grassi saturi in grammi

Tabella 6: Struttura di un pasto

8 Componenti del backend

8.1 Flusso di una richiesta a runtime

Quando il backend riceve una richiesta, esegue i seguenti passi in sequenza.

File	Ruolo
api/plan.php	Entry point: valida i parametri, avvia DietPlanner, restituisce JS
api/error.php	Funzione sendError(): risponde con JSON di errore e termina
src/Validator.php	Controlla e sanifica i parametri GET; lancia Exception se non val
src/DietPlanner.php	Costruisce il piano con logica anti-ripetizione e bilanciamento cal
src/RecipeRepository.php	Esegue le query SQL per ricette candidate e ingredienti
src/UsdaClient.php	Chiama USDA e salva i macro; restituisce i dati dalla cache se pr
src/IngredientNormalizer.php	Pulisce e normalizza i nomi degli ingredienti dal campo NER
src/Database.php	Singleton PDO per la connessione MySQL
setup/fetch_nutrition.php	Script da eseguire una volta per popolare i valori nutrizionali

Tabella 7: Componenti del backend e relativi ruoli

1. **Validazione:** Validator controlla tipi, range e valori ammissibili. In caso di errore restituisce subito HTTP 400 con JSON di errore.
2. **Selezione candidati:** per ogni pasto, RecipeRepository esegue una query SQL con i filtri di dieta, allergie e lista degli ID già usati, restituendo fino a 50 ricette ordinate casualmente.
3. **Scelta e scala:** DietPlanner sceglie la ricetta con meno ingredienti in comune con i pasti precedenti e calcola il portion_factor per avvicinarsi al target calorico del pasto.
4. **Bilanciamento giornaliero:** al termine di ogni giorno viene applicato un aggiustamento globale (clamped tra 0.7 e 1.5) per centrare esattamente il target calorico.
5. **Risposta:** la struttura dati viene serializzata in JSON e restituita con HTTP 200.

9 Logica no ripetizioni

L'obiettivo è garantire che nell'arco del piano settimanale non compaiano due volte la stessa ricetta né, per quanto possibile, lo stesso ingrediente principale.

Il sistema mantiene due liste durante la costruzione del piano: usedIds (ID delle ricette già scelte) e usedIngredients (nomi degli ingredienti già comparsi). La query SQL esclude sempre le ricette in usedIds; il metodo pick() privilegia poi le ricette senza ingredienti in usedIngredients. Se nessuna ricetta soddisfa entrambi i vincoli, viene comunque restituito il primo candidato disponibile per non bloccare la generazione del piano.

9.1 Distribuzione delle calorie

Le calorie giornaliere vengono ripartite tra i pasti secondo quote fisse. Quando il parametro snacks=1 è attivo, le quote vengono ridistribuite per includere uno spuntino.

Il portion_factor $s \in [0.5, 2.5]$ scala la ricetta sul target calorico del pasto. Tutti i valori nutrizionali vengono ricalcolati proporzionalmente. Un aggiustamento globale finale centra il totale giornaliero esattamente sul valore richiesto.

Pasto	Quota base	Quota con snack	Esempio (2000 kcal)
Colazione	25%	20%	500 / 400 kcal
Snack	—	10%	— / 200 kcal
Pranzo	40%	35%	800 / 700 kcal
Cena	35%	35%	700 / 700 kcal

Tabella 8: Distribuzione calorica per pasto

10 Routing con ModRewrite

Apache mod_rewrite riscrive le URL in ingresso verso il controller PHP principale. Il file .htaccess definisce tre regole in ordine:

1. Se il file richiesto esiste fisicamente, viene servito direttamente.
2. L'URL /api/plan viene reindirizzato a api/plan.php.
3. Qualsiasi altra rotta viene instradata a public/index.php (frontend).

Questo permette di esporre URL pulite nascondendo la struttura interna dei file PHP.

11 Testing

11.1 cURL

Il sistema può essere testato da linea di comando tramite cURL:

```
curl "http://localhost/cartella_progetto/public/index.php?calories=1700&days=4&allerg"
```

11.2 Postman

Per il testing tramite Postman è sufficiente:

1. Impostare il metodo **GET**.
2. Inserire l'URL del server XAMPP con il percorso /api/plan.
3. Aggiungere i parametri nella scheda **Params**.
4. Inviare la richiesta e verificare la risposta nella scheda **Body**.

12 Conclusioni

Il sistema realizzato espone un'API RESTful in PHP su XAMPP con le seguenti caratteristiche principali.

La separazione netta tra fase di importazione e fase di runtime elimina qualsiasi dipendenza dall'API USDA durante il normale funzionamento, riducendo i tempi di risposta da un range di 30–60 secondi (con chiamate API in tempo reale) a un massimo di 1–3 secondi (con accesso al solo database locale).

Il dataset Kaggle da oltre due milioni di ricette fornisce un catalogo ampio e variegato, mentre USDA FDC garantisce valori nutrizionali accurati e ufficiali su sette campi (calorie, proteine, grassi, carboidrati, fibre, zuccheri, grassi saturi). La cache locale elimina la dipendenza dal rate limit a runtime e può essere aggiornata manualmente con `–force-update`.

La logica per rimuovere le ripetizioni e la distribuzione calorica per pasto producono piani settimanali equilibrati e personalizzati. Il codice PHP è scritto con una sintassi semplice e accessibile, senza type hints avanzati né costrutti specifici di PHP 8.1+, per favorire la leggibilità e la manutenzione.